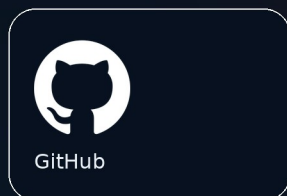
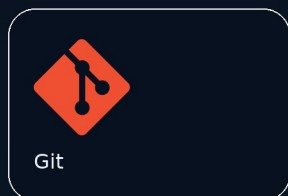


AMBIENTE E VERSIONAMENTO

# GIT FLOW NA PRÁTICA

Use o git-flow-next para organizar features, correções, releases e hotfixes sem fazer manualmente o trabalho que a ferramenta automatiza.



**Alexandre Carlos de Jesus**

[github.com/alecarlosjesus](https://github.com/alecarlosjesus)

[linkedin.com/in/alexandre-carlos-de-jesus](https://linkedin.com/in/alexandre-carlos-de-jesus)

Versões e procedimentos validados em agosto de 2026

© 2026 Alexandre Carlos de Jesus - Todos os direitos reservados.

ANTES DE COMEÇAR

# Como usar esta apostila

**ROTA DE ESTUDO** Leia o conceito, execute cada comando, compare a saída com o quadro Confira o resultado e somente depois avance. Digitar o comando sem observar o que aconteceu não conta como prática concluída.

ENTENDA OS TERMOS ANTES DE USÁ-LOS

## Palavras que você encontrará

| Termo         | Explicação em linguagem simples                                                                                          |
|---------------|--------------------------------------------------------------------------------------------------------------------------|
| Git Flow      | Modelo que define branches para desenvolvimento, versões e correções.                                                    |
| git-flow-next | Programa mantido em 2026 que executa e automatiza o modelo Git Flow.                                                     |
| main          | Branch permanente que representa as versões de produção.                                                                 |
| develop       | Branch permanente que reúne o trabalho destinado à próxima versão.                                                       |
| feature       | Branch temporária usada para desenvolver uma nova funcionalidade.                                                        |
| bugfix        | Branch temporária usada para corrigir um problema que ainda não é urgente em produção.                                   |
| release       | Branch temporária usada para preparar, testar e documentar uma versão.                                                   |
| hotfix        | Branch temporária usada para corrigir com urgência uma versão em produção.                                               |
| support       | Branch de manutenção prolongada de uma versão antiga. Em inglês, support significa suporte.                              |
| start         | Comando do git-flow-next que cria a branch correta e já muda o terminal para ela.                                        |
| publish       | Comando que envia uma branch temporária ao repositório remoto.                                                           |
| finish        | Comando que encerra o ciclo: integra a branch e, conforme a configuração, atualiza destinos, cria tag e remove a branch. |
| merge         | Operação que combina o histórico de uma branch com outra.                                                                |
| tag           | Nome fixo colocado em um commit para marcar uma versão, como v1.0.0.                                                     |
| origin        | Nome padrão dado ao repositório remoto principal.                                                                        |
| PATH          | Lista de pastas em que o sistema procura programas executáveis.                                                          |

O QUE SERÁ FEITO

## Organização da aula

| Item                          | Definição                                                                                                 |
|-------------------------------|-----------------------------------------------------------------------------------------------------------|
| Duração estimada              | 180 a 240 minutos                                                                                         |
| Pré-requisitos                | A02 obrigatória; A03 recomendada; repositório com branch main.                                            |
| Resultado esperado desta aula | Feature, bugfix, release v1.0.0 e hotfix v1.0.1 encerrados com git-flow-next; branches e tags conferidas. |
| Versões e base usadas         | git-flow-next 2.0.0 e modelo Classic GitFlow, consultados em agosto de 2026.                              |
| Arquivos trabalhados          | VERSION, docs/release-notes.md, .git/config e histórico de branches/tags                                  |

## AO FINAL DESTA AULA

## O que você aprenderá

Você conseguirá identificar a função de main, develop, feature, bugfix, release, hotfix e support.

Você conseguirá distinguir o modelo Git Flow da ferramenta git-flow-next.

Você conseguirá usar start, publish e finish sem repetir manualmente o trabalho da ferramenta.

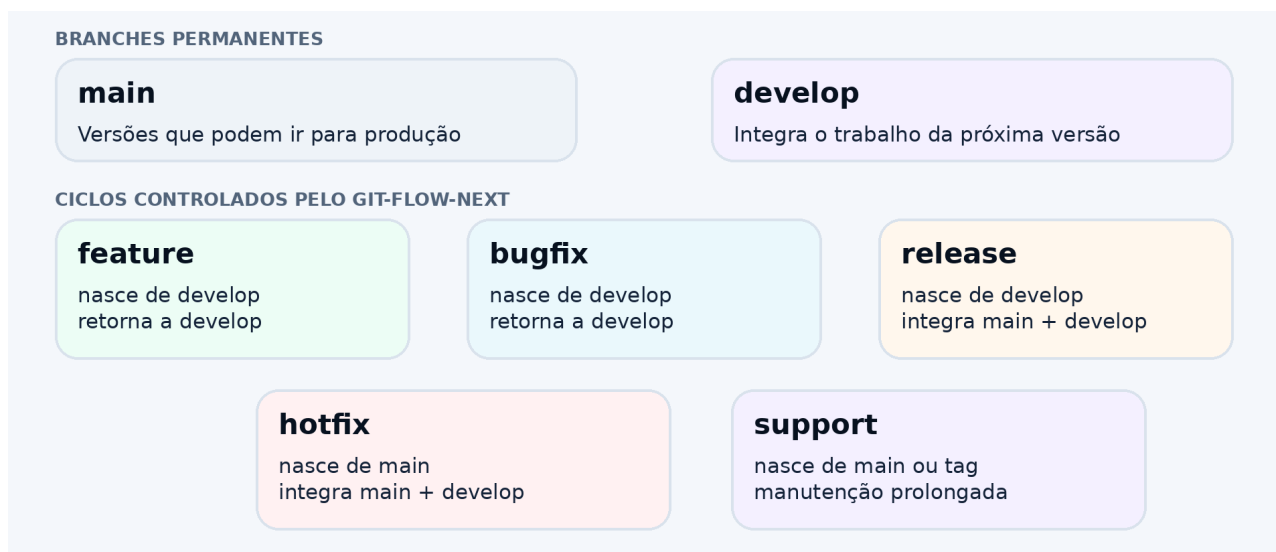
Você conseguirá conferir branches, merges e tags depois de cada encerramento.

Você conseguirá reconhecer o que o git-flow-next automatiza e o que ainda exige decisão humana.

Você conseguirá produzir a release v1.0.0 e o hotfix v1.0.1 do TechStore.

VEJA O ASSUNTO COMO UM TODO

## Como as partes se conectam



Mapa do Git Flow: main e develop são permanentes; feature, bugfix, release, hotfix e support têm origens e destinos definidos.

## MODELO E FERRAMENTA NÃO SÃO A MESMA COISA

## 1. Duas ideias com nomes parecidos

Git Flow é o modelo de organização das branches. Ele define onde uma branch começa, qual trabalho deve receber e para onde retorna. O git-flow-next é o programa que executa esse modelo por meio de comandos como start, publish e finish.

| Nome          | O que é                                                           | Exemplo                                              |
|---------------|-------------------------------------------------------------------|------------------------------------------------------|
| Git Flow      | Conjunto de regras para organizar branches.                       | Uma feature nasce de develop e retorna para develop. |
| git-flow-next | Ferramenta instalada no computador para automatizar essas regras. | git flow feature finish filtro-categoria             |

**ATENÇÃO** Nesta apostila o git-flow-next é a ferramenta principal. Os comandos manuais de merge e exclusão serão mostrados apenas para explicar o que acontece por baixo dos panos. Você não executará os dois procedimentos no mesmo ciclo.

**VOCÊ SABIA?** O Git Flow original e a edição gitflow-avh foram descontinuados. Em 2026 usamos o git-flow-next, implementação atual e compatível com a configuração antiga.

## PRIMEIRO ENTENDA O CAMINHO

## 2. O papel de cada branch

| Branch    | Nasce de    | Ao terminar                                           | Uso                                           |
|-----------|-------------|-------------------------------------------------------|-----------------------------------------------|
| feature/* | develop     | Retorna para develop                                  | Nova funcionalidade.                          |
| bugfix/*  | develop     | Retorna para develop                                  | Correção que ainda não é urgente em produção. |
| release/* | develop     | Integra main e atualiza develop                       | Preparação e teste final de uma versão.       |
| hotfix/*  | main        | Integra main e atualiza develop                       | Correção urgente de produção.                 |
| support/* | main ou tag | Permanece enquanto a versão antiga receber manutenção | Manutenção prolongada de uma versão antiga.   |

main e develop são permanentes. As demais branches existem para cumprir um objetivo. Feature, bugfix, release e hotfix normalmente são removidas quando o comando finish conclui o trabalho.

**ERRO COMUM** fix não é um tipo padrão de branch do Git Flow. Em mensagens de commit, fix: indica uma correção. Para branches, use bugfix quando a correção segue o desenvolvimento normal e hotfix quando a produção precisa ser corrigida com urgência.

## FERRAMENTA ATUAL PARA 2026

## 3. Instalar o git-flow-next

No Windows 10 ou 11, abra o PowerShell ou o Terminal do Windows. WinGet é o gerenciador de pacotes do Windows: ele localiza, baixa e instala programas por comando.

## WINDOWS - POWERSHELL OU TERMINAL

```
winget install GitTower.GitFlowNext
```

Quando a instalação terminar, feche e abra novamente o terminal. Essa reinicialização permite que o Windows atualize o PATH usado pela janela do terminal.

## VERIFICAÇÃO

```
git flow version
```

**CONFIRA O RESULTADO** O terminal exibe um número de versão seguido de (git-flow-next). Na documentação consultada em agosto de 2026, a versão publicada é 2.0.0.

**ERRO COMUM** Se aparecer 'flow is not a git command', o executável não foi encontrado. Reabra o terminal. Se o erro continuar, reinstale pelo WinGet ou confirme se git-flow.exe está em uma pasta registrada no PATH.

Em macOS, a instalação oficial pode ser feita com Homebrew: brew install gittower/tap/git-flow-next. Homebrew é um gerenciador de pacotes para macOS. Em Linux, consulte a página oficial de releases para o binário correspondente ao sistema.

## O INIT CRIA AS REGRAS DO FLUXO

## 4. Preparar e configurar o repositório

Use o repositório TechStore da A03. Antes de configurar o fluxo, confirme que você está na main e que não existem arquivos modificados sem commit.

## TERMINAL

```
git switch main
git pull --ff-only
git status
```

**CONFIRA O RESULTADO** A branch atual é main e o Git informa que não existem mudanças pendentes.

O comando abaixo usa o modelo clássico sem perguntas interativas. Também define os prefixos bugfix/, support/ e v para as tags de versão.

#### INICIALIZAR O GIT FLOW

```
git flow init --preset=classic --defaults --bugfix=bugfix/ --support=support/ --tag=v
```

preset significa configuração pronta. classic seleciona o Git Flow com main, develop, feature, release e hotfix. defaults aceita os nomes padrão. As opções restantes deixam explícitos os nomes que usaremos nesta aula.

#### CONFERIR A CONFIGURAÇÃO

```
git flow config list
git branch
```

**CONFIRA O RESULTADO** A configuração mostra main e develop, além dos tipos feature, bugfix, release, hotfix e support. A branch develop foi criada localmente.

O init configura o repositório local, mas não publica develop no GitHub. Por isso, somente nesta preparação inicial, envie a branch permanente:

#### PUBLICAR DEVELOP PELA PRIMEIRA VEZ

```
git push -u origin develop
```

#### TRÊS VERBOS PARA ACOMPANHAR

## 5. O ciclo start, publish e finish

| Comando        | O que a ferramenta faz                                                                                       | Quando usar                                          |
|----------------|--------------------------------------------------------------------------------------------------------------|------------------------------------------------------|
| <b>start</b>   | Cria a branch a partir da origem configurada e muda o terminal para ela.                                     | Ao começar um trabalho.                              |
| <b>publish</b> | Envia a branch temporária ao origin e cria o acompanhamento remoto.                                          | Quando a equipe precisa acessar ou revisar a branch. |
| <b>finish</b>  | Integra no destino, atualiza branches dependentes, cria tag quando configurado e remove a branch temporária. | Quando o trabalho está aprovado e pronto.            |

Usaremos duas opções no finish. --push envia as branches atualizadas e a tag ao origin. --no-keepremote solicita a remoção da branch temporária publicada. A branch local temporária já é removida por padrão.

#### EXEMPLO DO CICLO COMPLETO

```
git flow feature start filtro-categoria
git flow feature publish filtro-categoria
git flow feature finish filtro-categoria --push --no-keepremote
```

**ATENÇÃO** Não execute git branch -d depois de um finish concluído. A exclusão local já faz parte do finish. Primeiro leia a saída do comando e confira as branches.

#### DEVELOP -> FEATURE -> DEVELOP

## 6. Feature: criar uma funcionalidade

Nossa feature documentará um filtro por categoria. Como feature está configurada com develop como origem, não precisamos criar a branch manualmente nem escrever o prefixo feature/ no comando.

#### ETAPA 1 - INICIAR

```
git flow feature start filtro-categoria
git branch --show-current
```

**CONFIRA O RESULTADO** O terminal mostra feature/filtro-categoria como branch atual.

Abra docs/catalogo.md e acrescente uma seção explicando que o catálogo poderá ser filtrado por categoria. Salve o arquivo e registre a mudança:

## ETAPA 2 - IMPLEMENTAR

```
git status
git diff
git add docs/catalogo.md
git commit -m "feat: documenta filtro por categoria"
```

feat: indica que o commit adiciona uma funcionalidade. Esse prefixo pertence à mensagem do commit; não cria nem altera o tipo da branch.

## ETAPA 3 - PUBLICAR

```
git flow feature publish filtro-categoria
```

**CONFIRA O RESULTADO** A branch feature/filtro-categoria aparece no origin. develop ainda não recebeu o commit.

Depois da revisão e dos testes, encerre a feature com a própria ferramenta:

## ETAPA 4 - FINALIZAR

```
git flow feature finish filtro-categoria --push --no-keepremote
```

**CONFIRA O RESULTADO** A mudança foi integrada em develop; develop foi enviada ao origin; a branch temporária foi removida localmente e no remoto.

**VAMOS PRATICAR** Execute `git branch --all`. Procure feature/filtro-categoria. Ela não deve aparecer. Depois execute `git log --oneline --graph --decorate --all` e localize o commit da feature dentro de develop.

## CORREÇÃO SEM URGÊNCIA DE PRODUÇÃO

## 7. Bugfix: corrigir durante o desenvolvimento

Imagine que o link do catálogo está escrito incorretamente, mas essa alteração ainda não chegou à produção. Como o problema pertence à próxima versão, usamos bugfix e não hotfix.

## ETAPA 1 - INICIAR

```
git flow bugfix start corrige-link-catalogo
git branch --show-current
```

Corrija o texto ou o link em docs/catalogo.md. Depois registre, publique e finalize:

## ETAPAS 2 A 4

```
git add docs/catalogo.md
git commit -m "fix: corrige link do catalogo"
git flow bugfix publish corrige-link-catalogo
git flow bugfix finish corrige-link-catalogo --push --no-keepremote
```

**CONFIRA O RESULTADO** O commit de correção está em develop. main ainda representa a versão de produção anterior.

**VOCÊ SABIA?** bugfix é um tipo configurável no git-flow-next. Nesta aula ele foi preparado pelo comando `init` para nascer de develop. Equipes podem criar outros tipos, mas precisam documentar claramente sua origem e seu destino.

## DEVELOP -&gt; RELEASE -&gt; MAIN + DEVELOP

## 8. Release: preparar a versão 1.0.0

Release significa lançamento ou versão. Quando a equipe decide quais funcionalidades entrarão na versão 1.0.0, abre uma release. Nessa branch entram ajustes de versão, documentação e correções finais; não entram novas funcionalidades.

Usaremos versionamento semântico, no formato MAJOR.MINOR.PATCH:

| Parte | Quando aumenta                              | Exemplo        |
|-------|---------------------------------------------|----------------|
| MAJOR | Mudança incompatível com a versão anterior. | 1.4.2 -> 2.0.0 |
| MINOR | Nova funcionalidade compatível.             | 1.4.2 -> 1.5.0 |

| Parte | Quando aumenta       | Exemplo        |
|-------|----------------------|----------------|
| PATCH | Correção compatível. | 1.4.2 -> 1.4.3 |

**ETAPA 1 - INICIAR**

```
git flow release start 1.0.0
git branch --show-current
```

**CONFIRA O RESULTADO** A branch atual é release/1.0.0 e ela nasceu do estado atual de develop.

**CRIE OU ATUALIZE O ARQUIVO VERSION**

```
1.0.0
```

**DOCS/RELEASE-NOTES.MD**

```
# Release 1.0.0

## Entregas
- contrato inicial de Produto
- filtro por categoria
- correcao do link do catalogo

## Validacao
- documentacao revisada
- historico Git conferido
```

**ETAPA 2 - REGISTRAR E PUBLICAR**

```
git add VERSION docs/release-notes.md
git commit -m "chore: prepara release 1.0.0"
git flow release publish 1.0.0
```

chore: identifica uma tarefa de manutenção que não adiciona uma funcionalidade nem corrige diretamente um defeito de comportamento.

**ETAPA 3 - FINALIZAR**

```
git flow release finish 1.0.0 --message "release: TechStore 1.0.0" --push --no-keepremote
```

Na configuração clássica desta aula, o finish integra a release em main, cria a tag v1.0.0, atualiza develop com os ajustes finais, remove a branch release e publica os resultados.

**ETAPA 4 - CONFERIR**

```
git branch --all
git tag --list
git show v1.0.0 --stat
```

**CONFIRA O RESULTADO** A tag v1.0.0 existe em main; develop também contém os ajustes finais; release/1.0.0 não aparece na lista de branches.

MAIN -> HOTFIX -> MAIN + DEVELOP

## 9. Hotfix: corrigir a produção

Agora imagine que a versão v1.0.0 já está em produção e apresenta um texto de preço incorreto. develop pode conter mudanças ainda não lançadas. Por isso, a correção urgente precisa nascer de main.

**ETAPA 1 - INICIAR**

```
git flow hotfix start 1.0.1
git branch --show-current
```

**CONFIRA O RESULTADO** A branch atual é hotfix/1.0.1 e foi criada a partir de main.

Atualize VERSION para 1.0.1 e acrescente a correção em docs/release-notes.md. Depois:

**ETAPA 2 - REGISTRAR E PUBLICAR**

```
git add VERSION docs/release-notes.md
git commit -m "fix: corrige descricao de preco"
git flow hotfix publish 1.0.1
```

**ETAPA 3 - FINALIZAR**

```
git flow hotfix finish 1.0.1 --message "hotfix: TechStore 1.0.1" --push --no-keepremote
```

O finish integra a correção em main, cria a tag v1.0.1, atualiza develop, remove a branch hotfix e publica os resultados. Assim, a correção não desaparece da próxima versão.

#### ETAPA 4 - CONFERIR

```
git tag --list --sort=version:refname
git show v1.0.1 --stat
git log --oneline --graph --decorate --all
```

**CONFIRA O RESULTADO** As tags v1.0.0 e v1.0.1 aparecem; main e develop contêm a correção; hotfix/1.0.1 foi removida.

**ATENÇÃO** Se existir uma release aberta, confira se ela também recebeu o hotfix. Nenhuma linha futura pode perder uma correção que já foi aplicada em produção.

ASSUNTO AVANÇADO, MAS NECESSÁRIO

## 10. Support: manter uma versão antiga

Support significa suporte. Essa branch é usada quando a equipe precisa manter uma versão antiga enquanto a main continua avançando. Exemplo: clientes ainda usam a linha 1.x, mas o produto principal já está na linha 2.x.

#### EXEMPLO DE ABERTURA DA LINHA DE SUPORTE

```
git flow support start 1.x v1.0.1
git flow support publish 1.x
```

O nome 1.x representa a família de versões 1. O segundo argumento, v1.0.1, informa o ponto do histórico usado para iniciar a manutenção. Como support é uma linha prolongada, ela não deve ser encerrada somente para limpar a lista de branches.

**ATENÇÃO** Não execute finish em support durante esta prática. A equipe precisa decidir por quanto tempo a versão será mantida, como receberá correções e qual será o destino final. Esse acordo deve existir antes do encerramento da linha de suporte.

A FERRAMENTA PAUSA E O ALUNO DECIDE

## 11. O que acontece quando existe conflito

Conflito ocorre quando o Git não consegue combinar automaticamente mudanças feitas na mesma parte de um arquivo. O git-flow-next automatiza o processo, mas não adivinha qual conteúdo está correto.

#### RESOLVER E CONTINUAR

```
git flow feature finish nome-da-feature --push --no-keepremote
# se houver conflito, leia a mensagem e execute:
git status
# abra o arquivo, escolha o conteúdo correto e salve
git add caminho/do/arquivo
git flow feature finish nome-da-feature --continue
```

continue significa continuar uma operação que foi pausada. Se você perceber que começou o encerramento no momento errado, use o comando abaixo para cancelar a operação em andamento:

#### CANCELAR O FINISH EM ANDAMENTO

```
git flow feature finish nome-da-feature --abort
```

**ERRO COMUM** Não abra outro finish enquanto existe uma operação pausada. Primeiro resolva e use --continue, ou cancele com --abort. O git status ajuda a identificar o estado atual.

POR BAIXO DOS PANOS

## 12. O que o finish fez por você

| Ciclo   | Automação realizada nesta aula                                    | O aluno ainda precisa                                   |
|---------|-------------------------------------------------------------------|---------------------------------------------------------|
| feature | Integra em develop, remove a branch temporária e publica develop. | Criar o conteúdo, testar, revisar e resolver conflitos. |



| Ciclo   | Automação realizada nesta aula                                                         | O aluno ainda precisa                                 |
|---------|----------------------------------------------------------------------------------------|-------------------------------------------------------|
| bugfix  | Integra em develop, remove a branch temporária e publica develop.                      | Confirmar que a correção não é um hotfix de produção. |
| release | Integra em main, cria tag, atualiza develop, remove a release e publica os resultados. | Testar a versão e aprovar o lançamento.               |
| hotfix  | Integra em main, cria tag, atualiza develop, remove o hotfix e publica os resultados.  | Validar a urgência e testar a correção.               |
| support | Cria e publica a linha de manutenção.                                                  | Definir a política de manutenção e o encerramento.    |

**ATENÇÃO** --push envia as branches de destino e a tag. --no-keepremote pede a remoção da branch temporária remota. Se o GitHub bloquear uma operação por permissão ou proteção de branch, leia a mensagem: o trabalho local pode já ter sido concluído e precisará ser sincronizado conforme a regra da equipe.

NÃO CONFIE APENAS NA MENSAGEM DE SUCESSO

## 13. Conferência final e resumo de comandos

### DIAGNÓSTICO FINAL

```
git flow overview
git branch --all
git tag --list --sort=version:refname
git log --oneline --graph --decorate --all
git status
```

overview significa visão geral. O comando mostra a configuração e o estado do fluxo. Depois, a lista de branches confirma o que permaneceu, as tags mostram as versões e o gráfico permite acompanhar as integrações.

| Ciclo   | Iniciar                           | Publicar                       | Finalizar                                            |
|---------|-----------------------------------|--------------------------------|------------------------------------------------------|
| feature | git flow feature start nome       | git flow feature publish nome  | git flow feature finish nome --push --no-keepremote  |
| bugfix  | git flow bugfix start nome        | git flow bugfix publish nome   | git flow bugfix finish nome --push --no-keepremote   |
| release | git flow release start 1.0.0      | git flow release publish 1.0.0 | git flow release finish 1.0.0 --push --no-keepremote |
| hotfix  | git flow hotfix start 1.0.1       | git flow hotfix publish 1.0.1  | git flow hotfix finish 1.0.1 --push --no-keepremote  |
| support | git flow support start 1.x v1.0.1 | git flow support publish 1.x   | Somente quando a política de suporte determinar      |

**VAMOS PRATICAR** Explique oralmente o histórico sem olhar o gabarito: onde a feature começou, onde a release terminou, por que o hotfix nasceu de main e por que support não foi removida. Se alguma resposta não estiver clara, volte à seção correspondente.

VERIFIQUE O QUE APRENDEU

## Exercícios

Responda antes de consultar o gabarito. Nas questões práticas, registre o comando usado e a saída observada.

### 1. Quais são as duas branches permanentes do modelo usado nesta apostila?

Resposta: \_\_\_\_\_

**2. Explique a diferença entre Git Flow e git-flow-next.**

Resposta: \_\_\_\_\_

**3. Escreva o comando que inicia a feature chamada busca-produto.**

Resposta: \_\_\_\_\_

**4. O que o comando publish faz?**

Resposta: \_\_\_\_\_

**5. Depois de um feature finish concluído, por que não executamos git branch -d manualmente?**

Resposta: \_\_\_\_\_

**6. Um erro foi encontrado em uma funcionalidade que ainda está apenas em develop. Você usaria bugfix ou hotfix? Justifique.**

Resposta: \_\_\_\_\_

**7. O que o release finish realizou nesta prática?**

Resposta: \_\_\_\_\_

**8. Por que um hotfix nasce de main e precisa atualizar develop?**

Resposta: \_\_\_\_\_

**9. Para que serve uma branch support e por que ela não foi finalizada na prática?**

Resposta: \_\_\_\_\_

**10. Quais comandos devem ser usados depois de resolver os arquivos de um conflito durante feature finish?**

Resposta: \_\_\_\_\_

**11. Qual é a diferença entre o prefixo fix: em um commit e a branch hotfix?**

Resposta: \_\_\_\_\_

**12. Escreva a sequência git-flow-next para iniciar, publicar e finalizar a release 1.1.0, enviando resultados e removendo a branch remota.**

Resposta: \_\_\_\_\_

**APRENDA TAMBÉM COM A CORREÇÃO**

## Gabarito comentado

1. main, que representa produção, e develop, que integra o trabalho da próxima versão.
2. Git Flow é o modelo de organização das branches. git-flow-next é a ferramenta instalada que executa e automatiza esse modelo.
3. git flow feature start busca-produto. A ferramenta acrescenta o prefixo e cria feature/busca-produto a partir de develop.
4. publish envia a branch temporária ao origin e cria o acompanhamento remoto para que a equipe possa acessá-la.
5. Porque o finish já remove a branch local por padrão. Nesta prática, --no-keepremote também solicita a remoção remota. Repetir a exclusão é desnecessário e pode gerar erro.
6. bugfix, porque o problema está na linha de desenvolvimento e ainda não exige correção urgente da produção.
7. Integrou a release em main, criou a tag v1.0.0, atualizou develop, removeu a branch release e, com --push, publicou branches e tag.
8. Nascer de main evita carregar funcionalidades ainda não publicadas. Atualizar develop impede que a próxima versão perca a correção já aplicada em produção.
9. support mantém uma família antiga de versões. Ela permanece enquanto essa família receber manutenção e só deve terminar quando a política da equipe determinar.
10. Depois de corrigir e salvar o arquivo: git add caminho/do/arquivo; em seguida, git flow feature finish nome-da-feature --continue.
11. fix: é apenas uma convenção na mensagem do commit para indicar correção. hotfix é uma branch criada de main para resolver com urgência um problema de produção.
12. git flow release start 1.1.0; faça e registre os ajustes; git flow release publish 1.1.0; git flow release finish 1.1.0 --push --no-keepremote.

## ANTES DE ENCERRAR

## Lista de verificação final

- Consigo distinguir o modelo Git Flow da ferramenta git-flow-next;
- Consigo explicar a função de main, develop, feature, bugfix, release, hotfix e support;
- Consigo instalar e verificar o git-flow-next;
- Consigo usar start, publish e finish;
- Consigo finalizar feature e bugfix em develop;
- Consigo finalizar release e hotfix com atualização de main, develop e tags;
- Consigo resolver ou cancelar um finish com conflito;
- Consigo conferir branches, tags, estado e gráfico final;
- Consigo explicar o que a ferramenta automatiza sem repetir exclusões manuais.

## BIBLIOGRAFIA E ATUALIZAÇÃO

## Referências oficiais

- [git-flow-next - site oficial](#)
- [git-flow-next - instalação](#)
- [git-flow-next - comandos](#)
- [git-flow-next - configuração](#)
- [Vincent Driessen - modelo Git Flow original](#)
- [Repositório original - aviso de continuidade](#)
- [Pro Git - branches e merges](#)
- [Versionamento semântico](#)

*Documentação consultada e procedimentos validados em 11 de agosto de 2026.*

**AUTORIA E DIREITOS** | Alexandre Carlos de Jesus • GitHub: [alecarlosjesus](#) • LinkedIn: [alexandre-carlos-de-jesus](#) • © 2026  
Alexandre Carlos de Jesus. Todos os direitos reservados. Reprodução, distribuição, adaptação ou comercialização, total ou parcial, somente com autorização prévia por escrito, salvo permissões legais. Marcas e logotipos pertencem aos respectivos titulares.